01

Part 1 · Database Foundations

How databases keep data correct and fast, and how we model it for transactions and analytics.

- CRUD & ACID
- Atomicity & WAL / WAL Buffer
- Isolation levels & Indexes
- OLTP vs OLAP
- Normalization
- Database & Data Modeling
- Star & Snowflake Schema
- Slowly Changing Dimensions

1.1 CRUD Operations

★★☆

IN ONE LINE

Create, Read, Update, Delete — the four basic operations performed on persistent data.

CRUD is the vocabulary of every operational database and API. Data engineers care because **the same four verbs behave very differently** in an OLTP database (cheap, in-place, row-level) versus an analytical lake (immutable files, expensive rewrites).

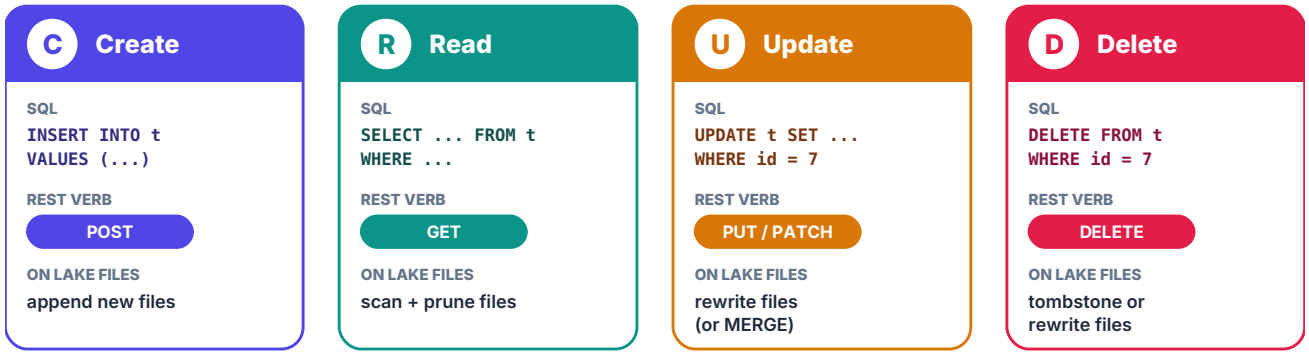


Figure 1.1 — CRUD across SQL, REST and file-based lakes

Op	SQL	Cost in OLTP DB	Cost on a data lake
Create	`INSERT`	Cheap — one row, one index update	Cheapest — just append a new Parquet file
Read	`SELECT`	Fast via index lookups	Fast if partitioned/clustered; scans otherwise
Update	`UPDATE`	Cheap — in-place (MVCC new version)	Costly — files are immutable, so rewrite the file (copy-on-write)
Delete	`DELETE`	Cheap — mark tuple dead, vacuum later	Costly — rewrite files, or write a *deletion vector*

INTERVIEW ANGLE

"Why are UPDATE and DELETE hard on a data lake?" Parquet files are immutable. Changing one row means rewriting the whole file that contains it. Table formats (Delta, Iceberg, Hudi) hide this behind transactions, and add tricks like **deletion vectors** / merge-on-read so you do not rewrite a 500 MB file to remove one row.

1.2 ACID Properties

★★★

IN ONE LINE

Atomicity, Consistency, Isolation, Durability — the four guarantees that make database transactions trustworthy.

ACID is the contract a relational database makes about **transactions** — a group of operations treated as one logical unit. It is the single most-asked database concept in interviews, usually followed by "how does the DB actually implement it?"

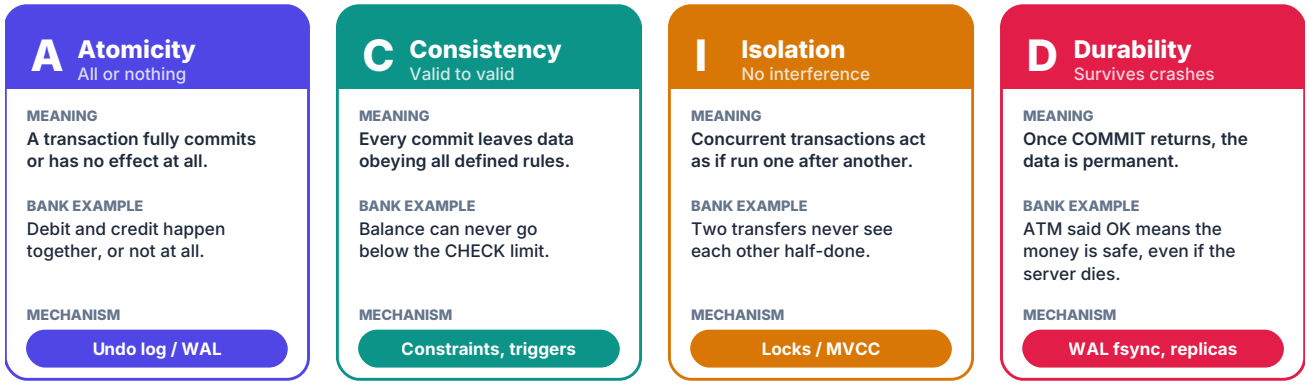


Figure 1.2 — The four ACID guarantees and the mechanism behind each

ACID → SQL / OLTP world

Strong guarantees, strict correctness. PostgreSQL, MySQL (InnoDB), Oracle, SQL Server. Also delivered on the lake by Delta Lake, Iceberg and Hudi.

BASE → many NoSQL systems

Basically **A**vailable, **S**oft state, **E**ventually consistent. Trades strictness for availability and horizontal scale (e.g. early Cassandra, DynamoDB eventual reads).

COMMON TRAP

The "C" in ACID (data obeys *application rules*) is **not** the "C" in the CAP theorem (all nodes see the same data). Interviewers love this mix-up. Also: consistency in ACID is partly the *application's* job — the DB only enforces rules you declared.

1.3 Atomicity (All-or-Nothing)

★★★

IN ONE LINE

A transaction either completes entirely or has no effect whatsoever.

Atomicity means a transaction is **indivisible**. If any step fails — constraint violation, crash, network drop, deadlock victim — the database undoes every change made so far so nobody ever observes a half-finished state.

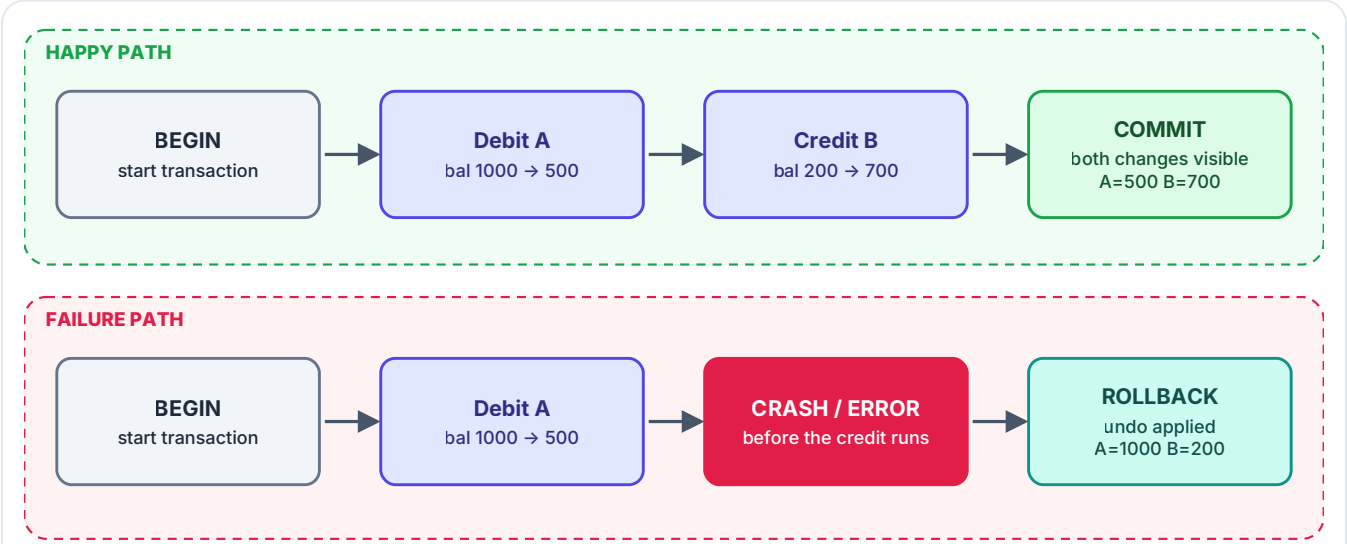


Figure 1.3 — Transfer 500 from account A to B: commit vs rollback

How is it implemented?

- **Undo information** — before changing a row the DB records enough to reverse it (undo log in MySQL/Oracle, old tuple versions in PostgreSQL MVCC).
- **Write-Ahead Log (next topic)** — every change is logged first. On crash recovery, uncommitted work is rolled back and committed work is replayed.
- **Commit as a single atomic step** — a transaction becomes visible only when its COMMIT record is durably in the log.

SQL · TRANSACTION

```sql
BEGIN;
  UPDATE accounts SET balance = balance - 500 WHERE id = 'A';
  UPDATE accounts SET balance = balance + 500 WHERE id = 'B';
COMMIT;   -- or ROLLBACK; to cancel everything
```

REMEMBER

Atomicity also shows up on the lake: a Delta Lake `MERGE` that fails halfway leaves **no partial files visible** because readers only trust files listed in the committed transaction log.

1.4 Write-Ahead Log (WAL) & WAL Buffer

★★★

IN ONE LINE

Changes are written to an append-only log *before* data files, so a crash can always be recovered or rolled back.

The core rule of WAL is simple: **log the change before you apply it to the data files**. Random writes to data pages are slow; sequentially appending a small log record is fast. So the database only has to make the *log* durable at commit time — the data pages can be flushed whenever convenient.

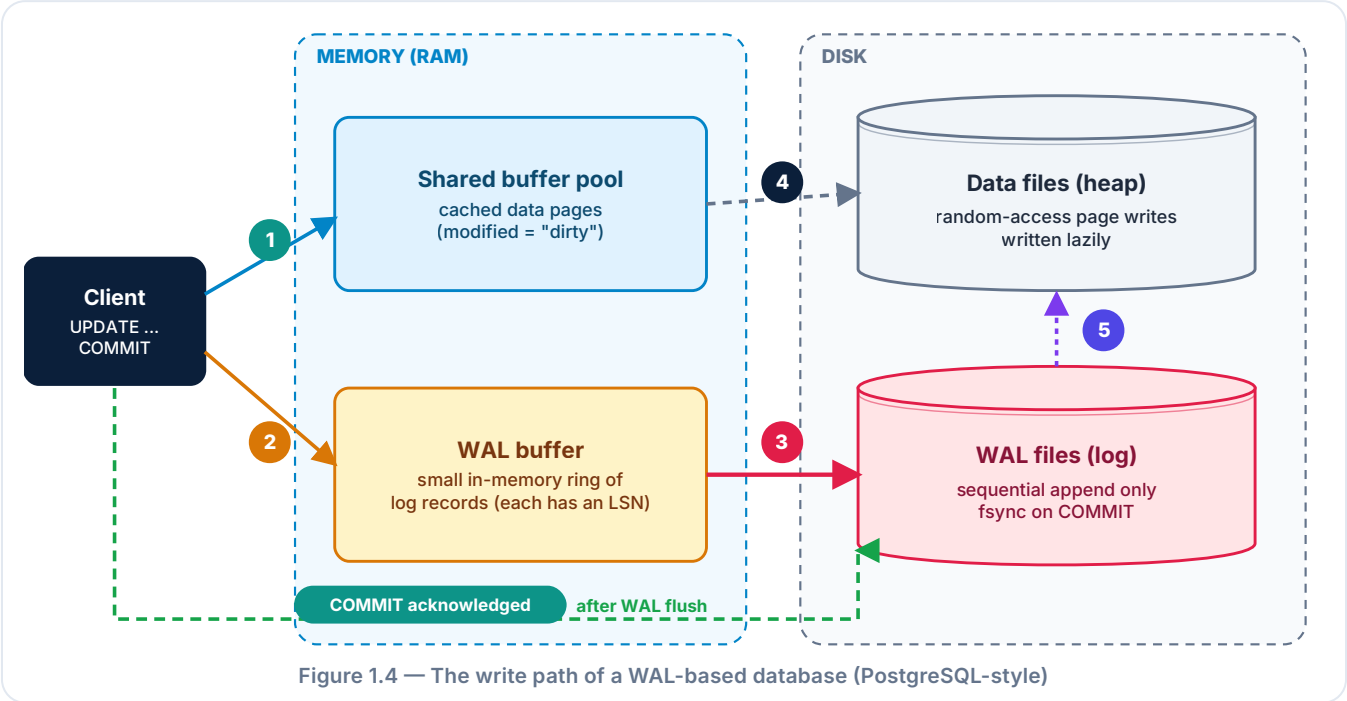


Figure 1.4 — The write path of a WAL-based database (PostgreSQL-style)

The five steps

1. Modify the page in the **shared buffer pool** (RAM) → it becomes a *dirty page*.
2. Append a log record describing the change to the **WAL buffer**; each record gets a Log Sequence Number (**LSN**).
3. On `COMMIT`, flush the WAL buffer to the **WAL file** and `fsync`. Only now is the client told "OK".
4. A **checkpoint** / background writer flushes dirty pages to data files later.
5. After a crash, **replay the WAL** from the last checkpoint (REDO) and discard uncommitted work.

Why it matters

- **Durability** — committed data survives power loss.
- **Atomicity** — uncommitted changes can be undone.
- **Performance** — sequential log writes, *group commit* batches many transactions into one fsync.
- **Replication** — replicas just stream and replay the WAL.
- **CDC** — tools like Debezium read the WAL (PostgreSQL) or binlog (MySQL) to capture changes with no load on tables.

REMEMBER

WAL buffer = the in-memory staging area for log records, flushed to disk at commit (or when full). Tuning it (e.g. `wal_buffers` in PostgreSQL) mainly helps write-heavy workloads with many concurrent commits.

INTERVIEW ANGLE

The same idea powers **Delta Lake's transaction log**, Kafka's commit log and LSM-tree databases. If asked "what is common between a database and Kafka?" — *both are built around an append-only log*.

1.5 Isolation Levels & MVCC

★★☆

IN ONE LINE

How much concurrent transactions are allowed to see of each other's in-flight work.

Perfect isolation (serializable) is expensive, so databases let you pick a weaker level and accept specific anomalies in exchange for concurrency.

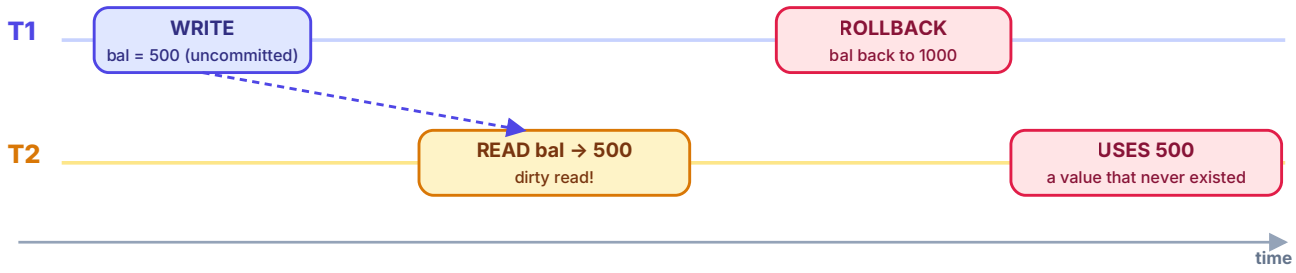


Figure 1.5 — A dirty read: T2 acts on data T1 later rolls back

Isolation level	Dirty read	Non-repeatable read	Phantom read	Typical default
Read Uncommitted	possible	possible	possible	rarely used
Read Committed	**prevented**	possible	possible	PostgreSQL, Oracle, SQL Server
Repeatable Read	prevented	**prevented**	possible*	MySQL InnoDB
Serializable	prevented	prevented	**prevented**	strictest, slowest

Anomalies

- **Dirty read** — reading uncommitted data.
- **Non-repeatable read** — same row read twice returns different values.
- **Phantom read** — same query returns new/missing *rows*.

How it's implemented

- **Locks** (2PL) — readers can block writers.
- **MVCC** — each transaction reads a snapshot; readers never block writers. Used by PostgreSQL, Oracle, InnoDB, and Delta Lake (snapshot isolation).

* PostgreSQL's Repeatable Read also prevents phantoms (snapshot isolation); the SQL standard only requires it at Serializable.

1.6 Indexes (B-Tree)

★★☆

IN ONE LINE

A sorted lookup structure that avoids full table scans at the cost of extra storage and slower writes.

An index is a separate, sorted structure that lets the database jump to rows instead of scanning the whole table. The default is the **B-tree** (balanced, sorted, great for equality and range lookups). Lookup cost is O(log n).

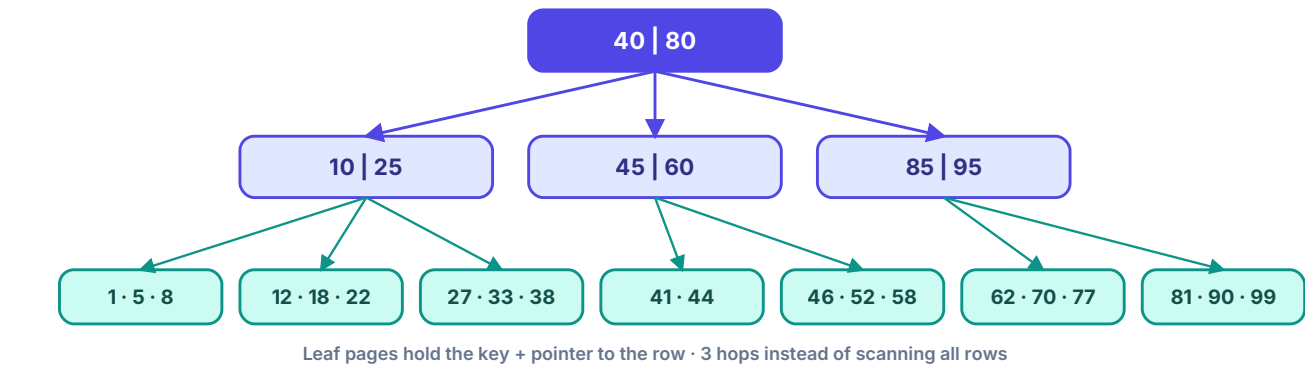


Figure 1.6 — B-tree: root, internal and leaf pages

Index type	Best for	Note
B-tree	equality, ranges, ORDER BY	Default almost everywhere
Hash	equality only	No range scans
Composite	multi-column filters	Column order matters (leftmost-prefix rule)
Covering	index-only scans	Index contains all needed columns
Bitmap / GIN	low-cardinality, arrays, text	Analytical / full-text use

COMMON TRAP

Indexes speed reads but **slow every write** (each INSERT/UPDATE must also update indexes) and consume storage. Column stores and lakehouses usually skip B-trees entirely and rely on **partition pruning, min/max statistics and clustering** instead.

1.7 OLTP vs OLAP (Row vs Column Storage)

★★★

IN ONE LINE

OLTP processes many small transactions; OLAP runs complex analytical queries over large historical data.

The two workload families that shape every data architecture. **OLTP** runs the business (many small, fast, concurrent transactions). **OLAP** analyses it (fewer, heavy, read-mostly queries scanning huge volumes). Their conflicting needs are why we copy data from one into the other.

	OLTP	OLAP
Purpose	Run day-to-day operations	Analytics, reporting, ML
Typical query	Insert/update one order; look up one customer	Revenue by region by month for 5 years
Rows touched	A handful	Millions to billions
Latency target	Milliseconds	Seconds to minutes
Schema	Normalised (3NF)	Denormalised (star / wide tables)
Storage layout	**Row-oriented**	**Column-oriented**
Concurrency	Thousands of small transactions	Tens of big queries
Examples	PostgreSQL, MySQL, Oracle, DynamoDB, Lakebase	Snowflake, BigQuery, Redshift, Databricks SQL, Trino

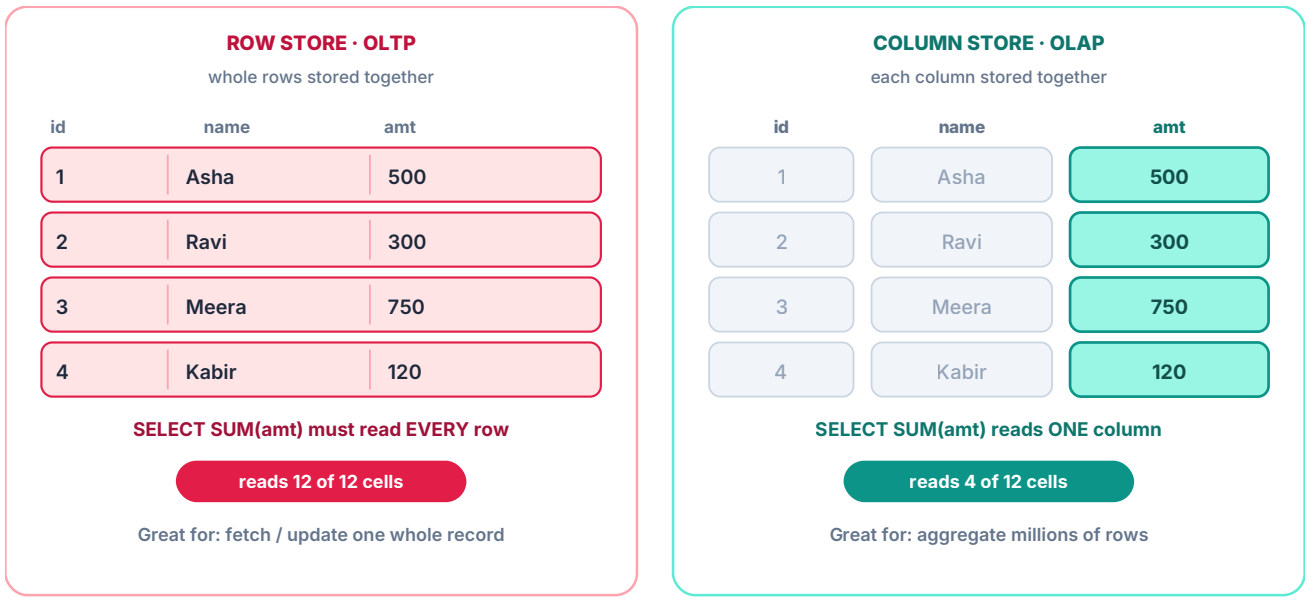


Figure 1.7 — Row vs column layout: why analytics engines read far less data

INTERVIEW ANGLE

Columnar layout also compresses much better (similar values sit together), enables **vectorised execution** and is the reason Parquet/ORC dominate analytics. A hybrid, **HTAP**, tries to serve both workloads in one system (e.g. TiDB, SingleStore).

1.8 Normalization (1NF → 3NF)

★★★

IN ONE LINE

Organising tables to remove redundancy and update anomalies by storing each fact once.

Normalisation removes redundancy by splitting data into tables so each fact lives in exactly **one place**. It prevents update, insert and delete anomalies and is the standard for OLTP schemas.

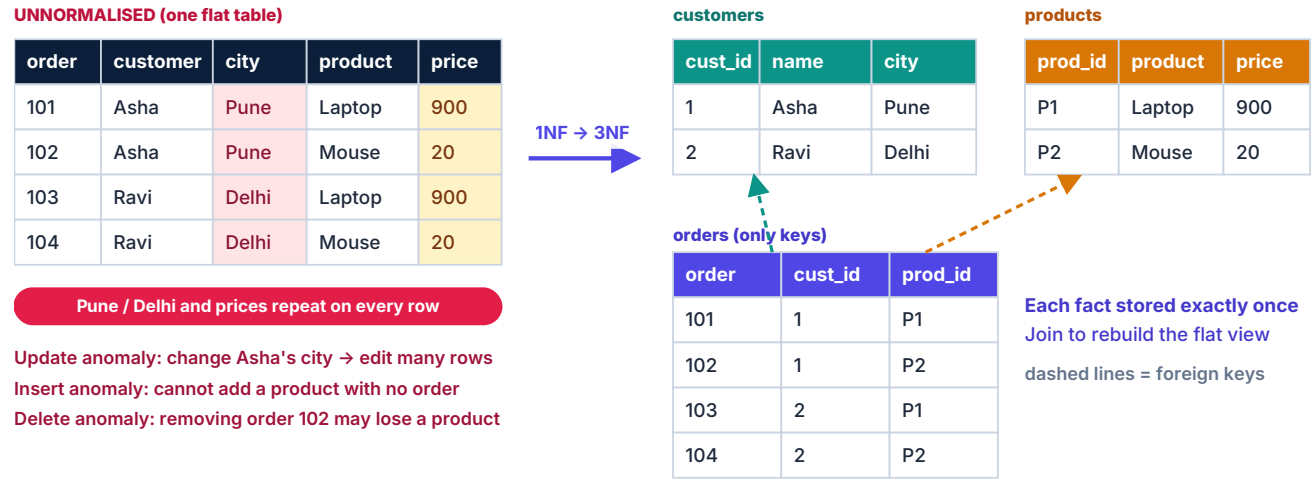


Figure 1.8 — From one repetitive table to three clean ones joined by keys

Form	Rule	Fix
1NF	Every cell is atomic; no repeating groups or arrays; each row unique	Split *"phones: 555,666"* into separate rows
2NF	1NF + no **partial** dependency (non-key column depends on only part of a composite key)	Move the partially-dependent columns to their own table
3NF	2NF + no **transitive** dependency (non-key column depends on another non-key column)	city → zip is a separate table, not on the customer
BCNF	Every determinant is a candidate key (stricter 3NF)	Rare edge cases with overlapping keys

INTERVIEW ANGLE

Memory hook: every non-key attribute must depend on **"the key, the whole key, and nothing but the key — so help me Codd."** Then add the twist: *analytics deliberately denormalises* (star schema, One Big Table) because joins are costly at scale and data is read-mostly.

1.9 Database Modeling & Data Modeling

★★★

IN ONE LINE

Designing how data is structured — from business concepts down to physical tables.

Database modelling usually means designing the schema of an operational (OLTP) system — ER diagrams, normalised tables, constraints. **Data modelling** is the wider discipline: shaping data for *any* purpose, including analytics (dimensional, Data Vault, wide tables). Both are done at three levels of detail.

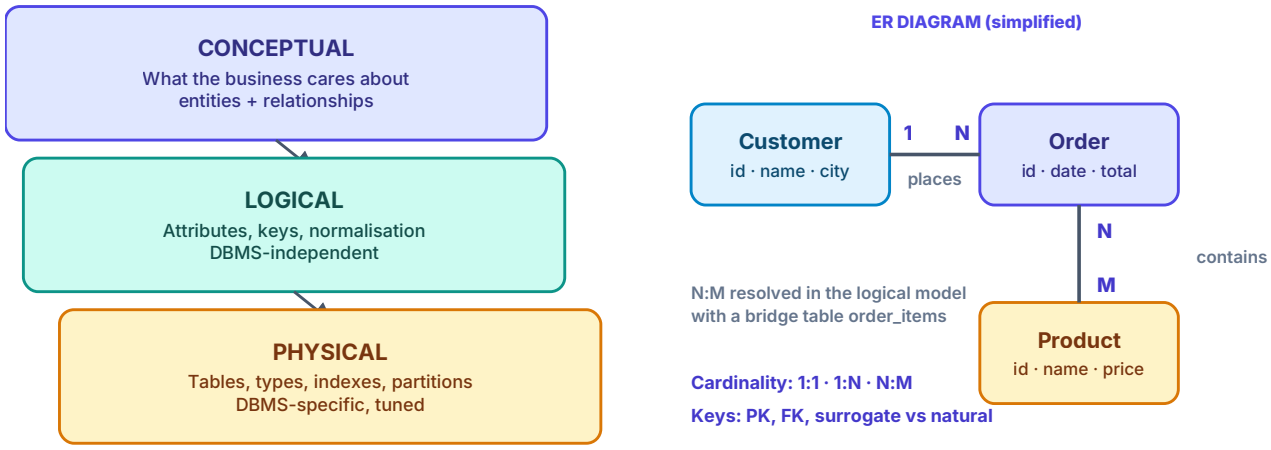


Figure 1.9 — Conceptual → logical → physical modelling, and a simple ER diagram

Approach	Where used	Shape	Strength
ER / 3NF	OLTP, Inmon-style EDW core	Many small normalised tables	Integrity, no redundancy
Dimensional (Kimball)	BI, marts, Gold layer	Fact + dimension tables	Fast, intuitive queries
Data Vault	Enterprise integration, auditability	Hubs, Links, Satellites	Agile, historised, parallel loads
One Big Table (OBT)	ML features, dashboards	One denormalised wide table	No joins at query time

INTERVIEW ANGLE

When asked "how would you model X?", clarify the **access pattern** first. OLTP → normalise. Analytics → pick the **grain**, then facts and dimensions. Naming the grain out loud is the single biggest signal of a senior modeller.

1.10 Dimensional Modeling: Star & Snowflake Schema

★★★

IN ONE LINE

Facts (measurements) surrounded by dimensions (context), designed around the business-process grain.

Dimensional modelling (Ralph Kimball) organises analytic data into **fact tables** (measurable events: sales, clicks, shipments) surrounded by **dimension tables** (descriptive context: who, what, where, when). The first design decision is always the **grain** — exactly what one row of the fact table represents.

Star schema

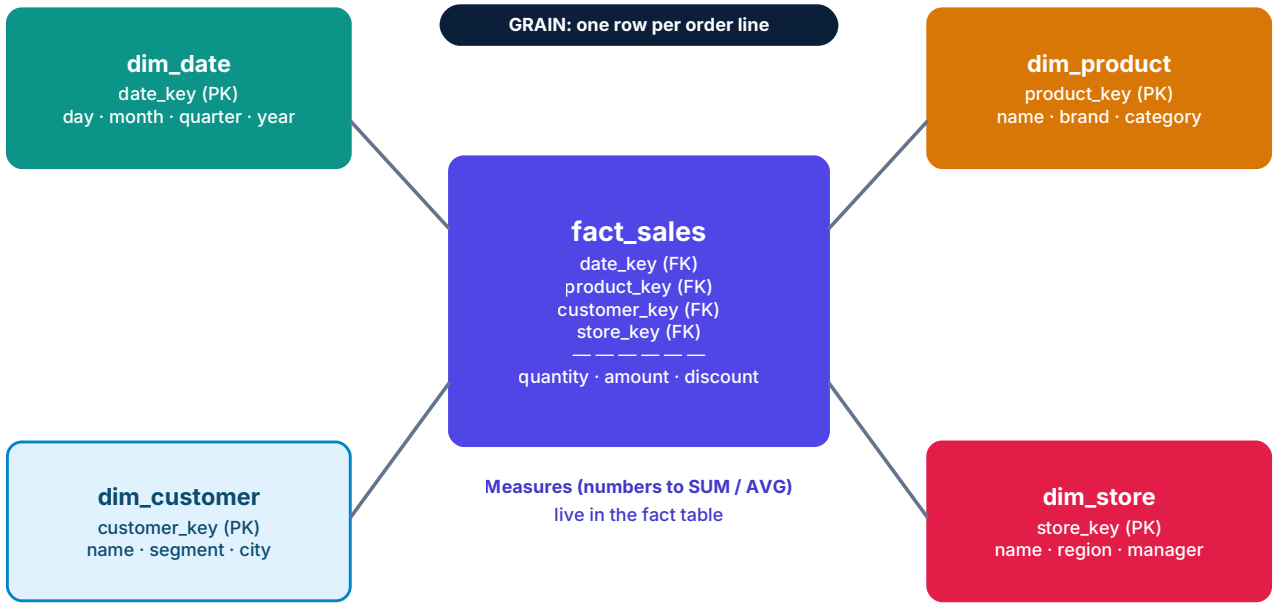


Figure 1.10a — Star schema: one fact table, denormalised dimensions one join away

Snowflake schema

Same idea, but dimensions are **further normalised** into sub-dimensions.

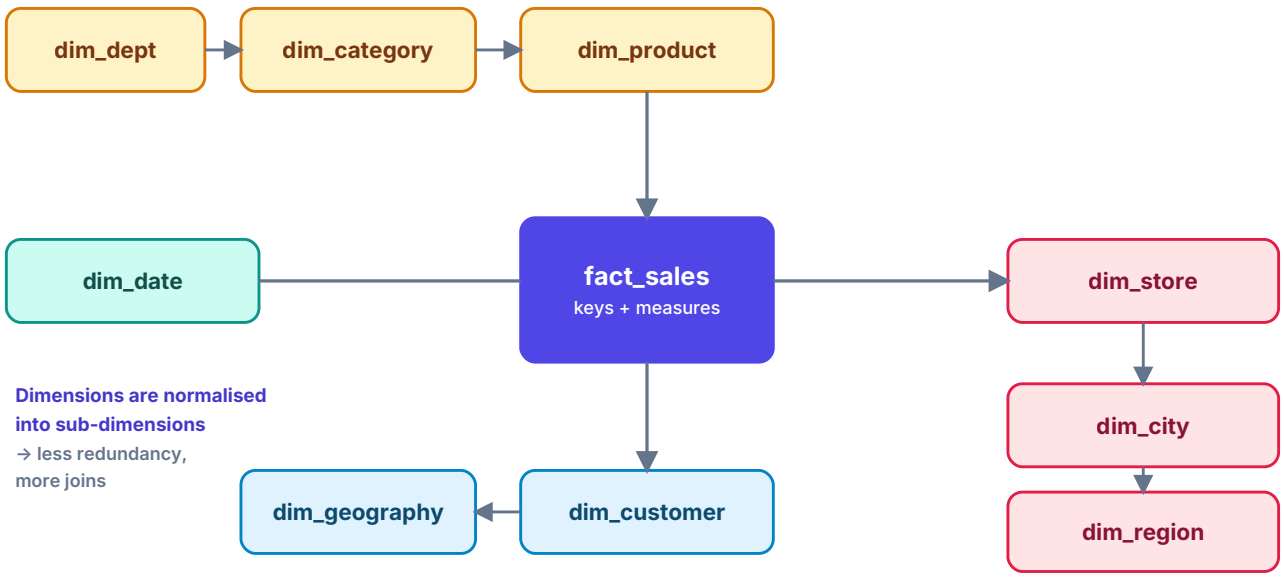


Figure 1.10b — Snowflake schema: dimensions split into hierarchies

	Star schema	Snowflake schema
Dimensions	Denormalised (flat)	Normalised (hierarchy of tables)
Joins per query	Fewer → faster	More → slower, more complex
Storage / redundancy	More redundancy	Less redundancy
Maintenance	Simple ETL, easy for BI users	Easier to update one attribute
Default choice?	**Yes** — storage is cheap, simplicity wins	Only for very large or shared dimensions

Vocabulary interviewers expect

Fact table types

- **Transaction** — one row per event.
- **Periodic snapshot** — one row per entity per period (daily balance).
- **Accumulating snapshot** — one row per process, updated as it progresses (order lifecycle).
- **Factless** — records that an event occurred (attendance).

Dimension concepts

- **Surrogate key** — meaningless integer PK, insulates from source-key changes.
- **Conformed** dimension shared across facts (same dim_date everywhere).
- **Role-playing** — dim_date as order date, ship date, due date.
- **Degenerate** dimension — an ID kept in the fact (order_number).
- **Junk** dimension — bundles low-cardinality flags.

REMEMBER

Measures are **additive** (revenue — sum across everything), **semi-additive** (account balance — sum across accounts, not across time) or **non-additive** (ratios, percentages — recompute from components).

1.11 Slowly Changing Dimensions (SCD)

★★★

IN ONE LINE

Techniques for tracking how descriptive attributes of a dimension change over time.

Dimension attributes change (a customer moves city, a product is re-categorised). **Slowly Changing Dimensions** define how history is handled. Example: customer *Asha* moves from Pune to Mumbai in July 2025.

TYPE 1 · OVERWRITE

id	name	city	
C1	Asha	Mumbai	← was Pune

No history kept. Simple; use for typo fixes.

TYPE 3 · PREVIOUS COLUMN

id	name	city	prev_city
C1	Asha	Mumbai	Pune

Only one prior value. Rarely used.

TYPE 2 · NEW ROW PER CHANGE (default for history)

surrogate_key	cust_id	name	city	valid_from	valid_to	is_current
101	C1	Asha	Pune	2023-01-01	2025-06-30	N
102	C1	Asha	Mumbai	2025-07-01	9999-12-31	Y

① **expire the old row (close valid_to, is_current = N)**
② **insert a new row with a NEW surrogate key (valid_to = 9999-12-31, is_current = Y)**

Fact rows keep pointing at the key that was current when they happened → "as-was" reporting.

Figure 1.11 — SCD Types 1, 2 and 3 for the same change

Type	Behaviour	History?	Use when
0	Never changes (retain original)	n/a	Birth date, original sign-up channel
1	Overwrite old value	× none	Corrections; history is irrelevant
2	Add a new row, version with dates / flag	✓ full	**Most common** — audit and point-in-time analytics
3	Add a column holding the previous value	◐ one step	Compare current vs prior only
4	Current table + separate history table	✓ full	Large, fast-changing dimensions
6	Hybrid 1 + 2 + 3	✓ full	Need current value on every historical row

Typical SCD2 load (Delta / SQL pattern)

SQL · SCD TYPE 2

```sql
-- 1) close rows whose tracked attributes changed
MERGE INTO dim_customer t
USING stg_customer s
  ON t.cust_id = s.cust_id AND t.is_current = true
WHEN MATCHED AND t.city <> s.city THEN
  UPDATE SET t.valid_to = current_date() - 1, t.is_current = false;

-- 2) insert new current versions (changed + brand-new customers)
INSERT INTO dim_customer
SELECT uuid(), s.cust_id, s.name, s.city, current_date(), DATE '9999-12-31', true
FROM stg_customer s
LEFT JOIN dim_customer t ON t.cust_id = s.cust_id AND t.is_current = true
WHERE t.cust_id IS NULL OR t.city <> s.city;
```

INTERVIEW ANGLE

Follow-up: *"How do you join a fact to an SCD2 dimension?"* — either by **surrogate key** captured at load time, or by natural key plus `fact.event_date BETWEEN valid_from AND valid_to`. dbt offers **snapshots**, and Lakeflow declarative pipelines offer `APPLY CHANGES INTO ... STORED AS SCD TYPE 2`, to automate this.